

Deliverable D1: Group Sheet and Grammar Specification

1. Group Information & Roll Numbers

- **Department:** Department of Computer Science, University of Engineering and Technology (UET), Lahore
- **Course:** CS-471L Compiler Construction Lab (Spring 2026)
- **Project:** Pascal Mini Compiler

Group Members:

1. **Abubakar Munir** (2023-CS-60) : **Group Leader**
 - *Role:* Developed the double-buffered Lexical Analyzer, the SLR(1) LR shift-reduce parser state machine, and the recursive symbol table manager.
2. **Ahmad Bin Rashid** (2023-CS-52)
 - *Role:* Designed and implemented the LL(1) Predictive Stack Parser with panic-mode synchronization, computed FIRST/FOLLOW sets, and error accumulator interface.
3. **Muhammad Omer** (2023-CS-68)
 - *Role:* Developed the Recursive Descent Parser, the Abstract Syntax Tree (AST) node hierarchy, AST visualization logic, and semantic validation rules.

2. Assigned Grammar in BNF Format

The formal context-free grammar (CFG) for our Pascal subset, designed to support declarations, functions, procedures, nested scopes, expressions, conditionals, and loops, is defined below in Backus-Naur Form (BNF).

```
Program      ::= "program" "id" ";" DeclPart CompoundStmt "."
DeclPart     ::= ConstDecl VarDecl ProcDecl
ConstDecl    ::= "const" ConstList | epsilon
ConstList    ::= "id" "=" ConstVal ";" ConstListTail
ConstListTail ::= "id" "=" ConstVal ";" ConstListTail | epsilon
ConstVal     ::= "num" | "string" | "true" | "false"

VarDecl      ::= "var" VarList | epsilon
VarList      ::= IdList ":" Type ";" VarListTail
VarListTail  ::= IdList ":" Type ";" VarListTail | epsilon
IdList       ::= "id" IdListTail
IdListTail   ::= "," "id" IdListTail | epsilon
Type         ::= SimpleType | ArrayType
SimpleType   ::= "integer" | "real" | "boolean" | "char"
ArrayType    ::= "array" "[" "num" ":" "num" "]" "of" SimpleType

ProcDecl     ::= ProcDeclProj ProcDecl | epsilon
ProcDeclProj ::= ProcedureHeader ";" DeclPart CompoundStmt ";"
               | FunctionHeader ";" DeclPart CompoundStmt ";"
ProcedureHeader ::= "procedure" "id" FormalParams
FunctionHeader  ::= "function" "id" FormalParams ":" SimpleType
FormalParams    ::= "(" FormalList ")" | epsilon
```

```

Formallist      ::= IdList ":" SimpleType FormallistTail
FormallistTail  ::= ";" IdList ":" SimpleType FormallistTail | epsilon

CompoundStmt   ::= "begin" StmtList "end"
StmtList       ::= Stmt StmtListTail
StmtListTail   ::= ";" Stmt StmtListTail | epsilon
Stmt           ::= SimpleStmt | StructuredStmt

SimpleStmt     ::= AssignOrCall | WriteStmt | ReadStmt | epsilon
AssignOrCall   ::= "id" AssignOrCallTail
AssignOrCallTail ::= "!=" Expr | "[" Expr "]" "!=" Expr | ActualParams | epsilon
ActualParams   ::= "(" ExprList ")" | epsilon
ExprList       ::= Expr ExprListTail
ExprListTail   ::= "," Expr ExprListTail | epsilon

WriteStmt      ::= "writeln" "(" ExprList ")" | "write" "(" ExprList ")"
ReadStmt       ::= "readln" "(" IdList ")" | "read" "(" IdList ")"

StructuredStmt  ::= CompoundStmt | IfStmt | WhileStmt | ForStmt
IfStmt         ::= "if" Expr "then" Stmt ElsePart
ElsePart       ::= "else" Stmt | epsilon
WhileStmt      ::= "while" Expr "do" Stmt
ForStmt        ::= "for" "id" "!=" Expr "to" Expr "do" Stmt

Expr           ::= SimpleExpr RelOpExpr
RelOpExpr      ::= RelOp SimpleExpr | epsilon
RelOp          ::= "=" | ">" | "<" | "!=" | ">" | "!="

SimpleExpr     ::= Term TermTail
TermTail       ::= AddOp Term TermTail | epsilon
AddOp          ::= "+" | "-" | "or"

Term           ::= Factor FactorTail
FactorTail     ::= MulOp Factor FactorTail | epsilon
MulOp          ::= "*" | "/" | "div" | "mod" | "and"

Factor         ::= "id" FactorIdentTail
               | "num"
               | "string"
               | "true"
               | "false"
               | "(" Expr ")"
               | "not" Factor
FactorIdentTail ::= "[" Expr "]" | ActualParams | epsilon

```

3. FIRST, FOLLOW, LL(1) and SLR(1) Tables

[!NOTE] All grammar tables (FIRST/FOLLOW sets, LL(1) Predictive parsing tables, and SLR(1) Action and Goto states) are computed programmatically by the compiled compiler driver to ensure mathematical accuracy.

The complete tables and state descriptions have been exported directly to: [parser tables.md](#) Please open and attach that file to complete this deliverable.

4. Sample Pascal Programs

Sample Program 1: Procedures, Functions, & Scopes (test/valid_program1.pas)

This program tests declarations, procedures, functions, parameters, local scope execution, and variable lookup resolution.

```
program ValidProg1;

const
  MAX = 100;
  PI = 3.1415;
  BANNER = 'Compiler Project Demo';

var
  x, y, result : integer;
  flag : boolean;

procedure greet(times : integer);
var
  i : integer;
begin
  for i := 1 to times do
  begin
    writeln(BANNER);
  end;
end;

function computeSquare(val : integer) : integer;
begin
  computeSquare := val * val;
end;

function checkGreater(a : integer; b : integer) : boolean;
begin
  if a > b then
    checkGreater := true
  else
    checkGreater := false;
end;

begin
  x := 10;
  y := 20;
  greet(3);

  if checkGreater(y, x) then
  begin
```

```

        result := computeSquare(y);
        writeln('Result is ', result);
    end
else
begin
    result := computeSquare(x);
    writeln('Result is ', result);
end;
end.

```

Sample Program 2: 1D Arrays & Control Loops (test/valid_program2.pas)

This program highlights one-dimensional array parsing, array index boundary checks, assignments to indexed memory, `while` loops, and `for` loops.

```

program ValidProg2;

var
    arr : array[1..10] of integer;
    i, sum : integer;

begin
    i := 1;
    while i := 10 do
    begin
        arr[i] := i * 5;
        i := i + 1;
    end;

    sum := 0;
    for i := 1 to 10 do
    begin
        sum := sum + arr[i];
    end;

    writeln('Calculated Array Sum: ', sum);
end.

```

Sample Program 3: User Inputs & Interactive Decisions (test/valid_program3.pas)

This program highlights reading input variables, checking constants, conditional logical decisions, and arithmetic operations.

```

program ValidProg3;

const
    THIRTY = 30;
    TRUEVAL = true;

var
    userInput, calc : integer;

```

```
flag : boolean;

begin
  write('Enter integer value: ');
  readln(userInput);

  if userInput < THIRTY then
    begin
      calc := userInput * 2;
      flag := TRUEVAL;
    end
  else
    begin
      calc := userInput - THIRTY;
      flag := false;
    end;
  end;

  writeln('Calculated result: ', calc);
  writeln('Flag condition: ', flag);
end.
```